

Multimodal Deep Learning

Solutions to Exercise Sheet 3

Date: 20th of May, 2025

Exercise 1: Checking permutation-equivariance

Let $\mathcal{S}_n$ be the permutation group on n indices and $P_\pi \in \{0, 1\}^{n \times n}$ the permutation matrix for permutation $\pi \in \mathcal{S}_n$. I.e. the permutation matrix is a square binary matrix that describes π , has exactly one entry of 1 in each row and each column, and 0s elsewhere. A function $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is *permutation-equivariant* iff

$$f(P_\pi x) = P_\pi f(x) \quad \forall x \in \mathbb{R}^n, \forall \pi \in \mathcal{S}_n.$$

For each of the following functions decide whether it is permutation-equivariant. Provide a short proof or a concrete counter-example (e.g. by exhibiting one $x \in \mathbb{R}^n$ and one non-trivial permutation π that violates the condition).

Throughout, let $x = (x_1, \dots, x_n)^\top$ with $n \geq 3$.

- (a) **Identity** $f_{\text{id}}(x) = x$.
- (b) **Element-wise non-linearity** $f_{\text{ReLU}}(x) = (\max\{0, x_1\}, \dots, \max\{0, x_n\})^\top$.
- (c) **Softmax vector** $f_{\text{soft}}(x) = \left(\frac{e^{x_1}}{\sum_{j=1}^n e^{x_j}}, \dots, \frac{e^{x_n}}{\sum_{j=1}^n e^{x_j}} \right)^\top$.
- (d) **Replicate first coordinate** $f_{\text{first}}(x) = (x_1, x_1, \dots, x_1)^\top$.
- (e) **Sorted output** $f_{\text{sort}}(x) = \text{sort}(x)$ (returns the coordinates in ascending order).
- (f) **Linear layer with shared scalar weight** $f_w(x) = w x$ for a fixed scalar $w \in \mathbb{R}$.
- (g) **Linear layer with dense weight matrix** $f_W(x) = Wx$ for a fixed *arbitrary* $W \in \mathbb{R}^{n \times n}$.

Hints.

- The definition demands *all* permutations. It suffices to find a single counter-example (x, π) to disprove equivariance.
- For (g) you may want to compare W with $P_\pi W P_\pi^\top$.

Solution

The condition for a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ to be permutation-equivariant is $f(P_\pi x) = P_\pi f(x)$ for all $x \in \mathbb{R}^n$ and for all permutations $\pi \in \mathcal{S}_n$. The i -th component of $P_\pi x$ is $x_{\pi^{-1}(i)}$. So the condition can be written component-wise as $(f(P_\pi x))_i = (f(x))_{\pi^{-1}(i)}$. We use $n \geq 3$.

- (a) $f_{\text{id}}(x) = x$. *Permutation-equivariant.*
Proof: $f_{\text{id}}(P_\pi x) = P_\pi x$. $P_\pi f_{\text{id}}(x) = P_\pi x$.
- (b) $f_{\text{ReLU}}(x) = (\max\{0, x_1\}, \dots, \max\{0, x_n\})^\top$. *Permutation-equivariant.*
Proof: Let $f = f_{\text{ReLU}}$. The i -th component of $f(P_\pi x)$ is $(f(P_\pi x))_i = \max\{0, (P_\pi x)_i\} = \max\{0, x_{\pi^{-1}(i)}\}$. The i -th component of $P_\pi f(x)$ is $(f(x))_{\pi^{-1}(i)} = \max\{0, x_{\pi^{-1}(i)}\}$.

- (c) $f_{\text{soft}}(x) = \left(\frac{e^{x_1}}{\sum_{j=1}^n e^{x_j}}, \dots, \frac{e^{x_n}}{\sum_{j=1}^n e^{x_j}} \right)^\top$. *Permutation-equivariant.*

Proof: Let $f = f_{\text{soft}}$ and $S(x) = \sum_{j=1}^n e^{x_j}$. The sum $S(P_\pi x) = \sum_{j=1}^n e^{(P_\pi x)_j} = \sum_{j=1}^n e^{x_{\pi^{-1}(j)}} = \sum_{k=1}^n e^{x_k} = S(x)$, as permuting terms does not change the sum. The i -th component of $f(P_\pi x)$ is $(f(P_\pi x))_i = \frac{e^{(P_\pi x)_i}}{S(P_\pi x)} = \frac{e^{x_{\pi^{-1}(i)}}}{S(x)}$. The i -th component of $P_\pi f(x)$ is $(f(x))_{\pi^{-1}(i)} = \frac{e^{x_{\pi^{-1}(i)}}}{S(x)}$.

- (d) $f_{\text{first}}(x) = (x_1, x_1, \dots, x_1)^\top$. *Not permutation-equivariant.*

Proof: Counter-example: Let $n = 3$, $x = (1, 2, 3)^\top$, and $\pi = (12)$. Then $P_\pi x = (2, 1, 3)^\top$. $f_{\text{first}}(P_\pi x) = (2, 2, 2)^\top$. $f_{\text{first}}(x) = (1, 1, 1)^\top$. So $P_\pi f_{\text{first}}(x) = P_\pi(1, 1, 1)^\top = (1, 1, 1)^\top$. $(2, 2, 2)^\top \neq (1, 1, 1)^\top$.

- (e) $f_{\text{sort}}(x) = \text{sort}(x)$. *Not permutation-equivariant.*

Proof: Counter-example: Let $n = 3$, $x = (3, 1, 2)^\top$. Then $f_{\text{sort}}(x) = (1, 2, 3)^\top$. Let $\pi = (12)$. $P_\pi x = (1, 3, 2)^\top$. $f_{\text{sort}}(P_\pi x) = f_{\text{sort}}((1, 3, 2)^\top) = (1, 2, 3)^\top$. $P_\pi f_{\text{sort}}(x) = P_\pi(1, 2, 3)^\top = (2, 1, 3)^\top$. Since $(1, 2, 3)^\top \neq (2, 1, 3)^\top$, the function is not permutation-equivariant.

- (f) $f_w(x) = w x$. *Permutation-equivariant.*

Proof: $f_w(P_\pi x) = w(P_\pi x)$. $P_\pi f_w(x) = P_\pi(w x) = w(P_\pi x)$, since matrix multiplication is linear and scalar multiplication commutes with matrix multiplication.

- (g) $f_W(x) = W x$. *Not permutation-equivariant (unless W has a specific structure).*

Counter-example: Let $n = 3$, $W = \text{diag}(1, 2, 3) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}$. Let $x = (0, 1, 0)^\top$.

Let π be the permutation swapping 1 and 2. $P_\pi = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$. So $P_\pi x = (1, 0, 0)^\top$.

$f_W(P_\pi x) = W(P_\pi x) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix} (1, 0, 0)^\top = (1, 0, 0)^\top$. $P_\pi f_W(x)$. First, $f_W(x) =$

$W x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix} (0, 1, 0)^\top = (0, 2, 0)^\top$. Then $P_\pi(0, 2, 0)^\top = (2, 0, 0)^\top$. Since $(1, 0, 0)^\top \neq (2, 0, 0)^\top$, the function is not permutation-equivariant for this W .

Exercise 2: Attention Masks

Correction Note: The previous version of this document contained an error in its description of how binary masks are applied. It incorrectly stated that the mask is multiplied with the attention scores. This is false; multiplying a score by 0 before a softmax operation would result in $\exp(0) = 1$, which fails to suppress the attention weight. The correct method is to use an additive mask, as described below. A binary 0,1 mask serves as a conceptual tool for defining allowed connections, which is then either converted to an additive mask for application or used to prevent computation altogether in optimized code.

Attention masks can be represented in two main ways:

- **Binary attention masks:** $M \in \{0, 1\}^{n \times n}$, where $M_{ij} = 1$ if token i (query) is allowed to attend to token j (key), and $M_{ij} = 0$ otherwise. This is a conceptual representation used to define a policy.

- **Additive attention masks:** Here, the mask values are added to the attention logits (scores before softmax). Positions where attention is allowed are assigned a value of 0, while disallowed positions are assigned a large negative value (e.g., $-\infty$ or a very small number like -10^9). This ensures that after the softmax operation, the probabilities for disallowed positions become zero.

In Parts (a)-(c) of this exercise, you will design *binary attention masks*. In Part (d), you will design an *additive attention mask*. For all binary masks, use the convention $M_{ij} = 1 \iff$ token i is allowed to attend to token j .

(a) **Causal sequence (autoregressive) mask**

Consider a sequence of length $n = 8$ processed left-to-right.

1. Write down the 8×8 binary causal mask M^{causal} that allows each position i to attend only to positions $k \leq i$ (including itself).
2. Give a closed-form expression for M_{ij}^{causal} (for a binary mask) for general sequence length n .

(b) **Binary tree mask**

A binary tree with seven nodes is stored in an array using breadth-first order (BFS), i.e., the tokens are [node0, node1, ..., node6] corresponding to [root, L, R, LL, LR, RL, RR] where L/R denote left/right children. The tree structure is shown in Figure 1. Design a binary mask $M^{\text{tree}} \in \{0, 1\}^{7 \times 7}$ that fulfils *all* of the following conditions:

- A node may attend to its parent.
- A node may attend to its two children (if they exist).
- A node may also attend to itself.
- No other attention is allowed.

Write down the 7×7 matrix M^{tree} .

Optionally, derive a general formula for M_{ij}^{tree} using integer arithmetic on i, j for a complete binary tree of N nodes stored in BFS order.

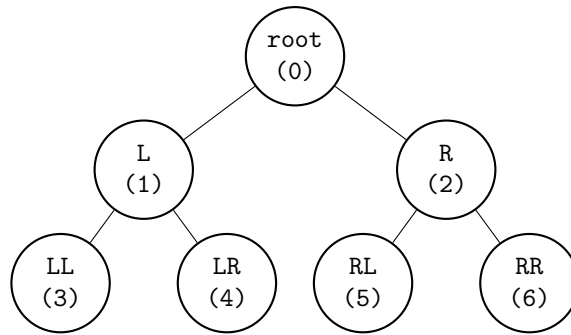


Figure 1: Binary tree structure for Part (b). Nodes are indexed 0-6 in Breadth-First Order (BFS). 'L' denotes left child, 'R' denotes right child.

(c) **Special datatype: heterogeneous tabular row**

A single table row is tokenised into a sequence of length $1 + d_n + d_c$:

$$[\text{CLS}, x_1^{\text{num}}, \dots, x_{d_n}^{\text{num}}, x_1^{\text{cat}}, \dots, x_{d_c}^{\text{cat}}],$$

where the first token **CLS** is a learned global representation, the d_n numeric features are x_i^{num} , and the d_c categorical features are x_i^{cat} .

Design a binary mask M^{tab} that enforces the following inductive biases:

1. The **CLS** token can attend to every other token in the row, and every other token can attend to **CLS**.
2. Numeric tokens x_i^{num} can *only* attend to the **CLS** token and to *any* numeric token (including themselves). They cannot attend to categorical tokens.
3. Categorical tokens x_i^{cat} can *only* attend to the **CLS** token and to *any* categorical token (including themselves). They cannot attend to numeric tokens.

Let the token indices be 0 for **CLS**; $1, \dots, d_n$ for numeric tokens; and $d_n + 1, \dots, d_n + d_c$ for categorical tokens.

1. Describe the general block structure of M^{tab} for arbitrary $d_n \geq 1, d_c \geq 1$. You can use **1** to denote a submatrix of all ones and **0** for a submatrix of all zeros.
2. Provide the specific 4×4 binary mask matrix for the case $(d_n, d_c) = (1, 2)$. The tokens are $[\text{CLS}, x_1^{\text{num}}, x_1^{\text{cat}}, x_2^{\text{cat}}]$.

(d) **Graph neighbourhood mask**

Consider an undirected graph with $N = 5$ nodes and an adjacency matrix $A \in \{0, 1\}^{N \times N}$ (where $A_{ij} = 1$ if node i and node j are connected, and $A_{ij} = 0$ otherwise; assume $A_{ii} = 0$ for all i , i.e., no self-loops in A itself). You want each node to attend only to itself and its direct neighbors in the graph.

1. Provide the general rule to construct an *additive attention mask* M^{graph} from the adjacency matrix A .
2. Given the adjacency matrix A for $N = 5$:

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

Write down the corresponding 5×5 additive mask matrix M^{graph} .

Optional Discuss how your masks (causal, tree, tabular) could be extended or combined if a model processes several independent structures (e.g., multiple sequences, trees, or table rows) *batched* together. Assume the global input sequence is a concatenation of these independent structures, possibly separated by special **[SEP]** tokens. What is the main principle for masking in such batched scenarios to maintain independence between batch items?

Solution

- (a) 1. The 8×8 binary causal mask M^{causal} for a sequence of length $n = 8$ (assuming 0-indexed positions $0, \dots, 7$) is:

$$M^{\text{causal}} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

2.

$$M_{ij}^{\text{causal}} = \begin{cases} 1, & j \leq i, \\ 0, & j > i, \end{cases} \quad 0 \leq i, j < n.$$

- (b) The nodes are indexed $0, \dots, 6$ in BFS order. The 7×7 binary tree mask M^{tree} is:

$$M^{\text{tree}} = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

- (c) Token indices are 0 for CLS; $1, \dots, d_n$ for numeric; $d_n + 1, \dots, d_n + d_c$ for categorical.

1. The general block structure of M^{tab} for $d_n \geq 1, d_c \geq 1$ is:

$$M^{\text{tab}} = \begin{pmatrix} 1 & \mathbf{1}_{1 \times d_n} & \mathbf{1}_{1 \times d_c} \\ \mathbf{1}_{d_n \times 1} & \mathbf{1}_{d_n \times d_n} & \mathbf{0}_{d_n \times d_c} \\ \mathbf{1}_{d_c \times 1} & \mathbf{0}_{d_c \times d_n} & \mathbf{1}_{d_c \times d_c} \end{pmatrix}$$

Here, $\mathbf{1}_{r \times c}$ is an $r \times c$ matrix of ones, and $\mathbf{0}_{r \times c}$ is an $r \times c$ matrix of zeros. The blocks correspond to CLS (1 row/col), numeric (d_n rows/cols), and categorical (d_c rows/cols) sections.

2. For $(d_n, d_c) = (1, 2)$, the tokens are $[\text{CLS}, x_1^{\text{num}}, x_1^{\text{cat}}, x_2^{\text{cat}}]$. The 4×4 binary mask M^{tab} is:

$$M^{\text{tab}} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

- (d) 1. Let A be the $N \times N$ adjacency matrix where $A_{ii} = 0$. Each node i attends to itself and its direct neighbors. The additive attention mask M^{graph} is constructed as:

$$M_{ij}^{\text{graph}} = \begin{cases} 0 & \text{if } i = j \text{ or } A_{ij} = 1 \\ -\infty & \text{otherwise} \end{cases}$$

This can also be written as $M_{ij}^{\text{graph}} = (-\infty) \cdot (1 - (A + I)_{ij})$, where I is the $N \times N$ identity matrix and $(A + I)_{ij}$ is treated as a binary indicator.

2. Given the adjacency matrix $A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{pmatrix}$: The corresponding 5×5 additive

mask M^{graph} is:

$$M^{\text{graph}} = \begin{pmatrix} 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & -\infty & 0 & 0 \\ 0 & -\infty & 0 & -\infty & -\infty \\ -\infty & 0 & -\infty & 0 & 0 \\ -\infty & 0 & -\infty & 0 & 0 \end{pmatrix}$$

Exercise 3: Positional Encodings for Special Datatypes

Definition: Sinusoidal Positional Encodings

Sinusoidal positional encodings are a type of fixed (non-learned) positional encoding used in Transformer models to inject information about the position of tokens in a sequence. For a token at position ‘pos’ in a sequence and a model of dimension d_{model} , the positional encoding vector $\text{PE}(\text{pos}) \in \mathbb{R}^{d_{\text{model}}}$ is defined as follows:

For each dimension j from 0 to $d_{\text{model}} - 1$:

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \\ \text{PE}(\text{pos}, 2i + 1) &= \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \end{aligned}$$

where $j = 2i$ or $j = 2i + 1$. Here:

- pos is the position of the token in the input sequence (e.g., $0, 1, 2, \dots$).
- i is the index for the wavelength pair (ranging from 0 to $\lfloor (d_{\text{model}} - 1)/2 \rfloor$).
- d_{model} is the dimensionality of the embeddings (and thus of the PE vector).

In this exercise you will design positional encodings for data types that are *not* plain 1-D sequences.

Let d be the model dimension and assume all positional vectors are in $\mathbb{R}^d$.

- (a) **Binary tree** Consider the complete binary tree of depth 2 (7 nodes), as shown in Figure 1. The nodes are stored in an array using breadth-first order (BFS): indices $0, \dots, 6$ correspond to [root, L, R, LL, LR, RL, RR].

1. Propose a *two-dimensional* index ($\text{depth}(v), \text{offset}(v)$) for every node v and embed it via the 2-D sinusoidal scheme

$$\text{PE}_{\text{tree}}(v) = \text{PE}_{1\text{D}}(\text{depth}(v)) + \text{PE}_{1\text{D}}(\text{offset}(v)),$$

where $\text{PE}_{1\text{D}}$ is the sine/cosine encoding.

2. Write down the $7 \times d$ matrix of positional vectors for $d = 4$.
3. Generalise your formula to an arbitrary b -ary tree of depth L .

- (b) **2-D grid (image patches)** Following the ViT patchification from the lecture, an image of $H \times W$ pixels is split into $(H/p) \times (W/p)$ patches. Let the patch at row r , column c have the encoding

$$\text{PE}_{2\text{D}}(r, c) = \text{PE}_{1\text{D}}(r) + \text{PE}_{1\text{D}}(c).$$

1. Explain why the row/column information remains disentangled.
2. Show how to include a learned class token [CLS] that attends globally.

- (c) **Multi-variate time series (temporal + variate)** A sensor signal consists of V variates (channels), and each variate's time series is cut into T non-overlapping patches (or segments). This yields T time steps, each having V associated variate values.

A positional encoding scheme for a token corresponding to variate $v \in \{0, \dots, V-1\}$ at time step t is:

$$\text{PE}_{\text{signal}}(t, v) = \text{PE}_{1\text{D}}(t) + \mathbf{e}_v^{\text{var}},$$

where $\text{PE}_{1\text{D}}(t)$ is the standard sinusoidal encoding for time step t , and $\mathbf{e}_v^{\text{var}} \in \mathbb{R}^d$ is a learned embedding for variate v .

1. Explain how this additive scheme allows the model to potentially factorise information about "when" an event occurs (temporal aspect) and "on which channel/variante" it occurs.
2. Discuss one potential drawback of this simple additive scheme, especially when dealing with variates that have complex, time-dependent correlations or phase shifts relative to each other.

Optional Contrast absolute encodings (as above) with *relative* or *rotary* schemes (slide 104) and argue when they would be preferable for very deep trees or extremely long signals.

Solution

- (a) **Binary tree (depth = 2, 7 nodes)**

For every node v let

$$(\text{depth}(v), \text{offset}(v)) \quad \text{with} \quad \text{depth}(v) \in \{0, 1, 2\}, \quad \text{offset}(v) \in \{0, \dots, 2^{\text{depth}(v)} - 1\}.$$

Nodes at the same depth are counted left to right. The positional vector is obtained by an additive 2-D scheme

$$\text{PE}_{\text{tree}}(v) = \text{PE}_{1\text{D}}(\text{depth}(v)) + \text{PE}_{1\text{D}}(\text{offset}(v))$$

With $d_{\text{model}} = 4$ we have

$$\text{PE}_{1\text{D}}(x) = [\sin x, \cos x, \sin(x/100), \cos(x/100)].$$

In breadth-first order [root, L, R, LL, LR, RL, RR]:

node	$j = 0$	$j = 1$	$j = 2$	$j = 3$
root (0, 0)	0	2.0000	0.0000	2.0000
L (1, 0)	0.8415	1.5403	0.0100	1.9999
R (1, 1)	1.6829	1.0806	0.0200	1.9999
LL (2, 0)	0.9093	0.5839	0.0200	1.9998
LR (2, 1)	1.7508	0.1242	0.0300	1.9998
RL (2, 2)	1.8186	-0.8323	0.0400	1.9996
RR (2, 3)	1.0504	-1.4061	0.0500	1.9994

- *Depth index*: $\text{depth}(v) \in \{0, \dots, L\}$.
- *Offset index*:

$$\text{offset}(v) = \sum_{k=0}^{\text{depth}(v)-1} b^k c_k, \quad c_k \in \{0, \dots, b-1\},$$

where c_k is the k -th base- b digit of the path (0 = “leftmost child”).

(b) **2-D grid (image patches)**

Disentanglement of row/column. The exercise was misleadingly phrased.

In fact we wanted you to observe that the proposed additive scheme, $\text{PE}_{2D}(r, c) = \text{PE}_{1D}(r) + \text{PE}_{1D}(c)$, where PE_{1D} is the standard 1D sinusoidal encoding applied to both row and column indices, has a notable limitation: it does not produce unique encodings for all distinct (r, c) pairs. Specifically, due to the commutativity of addition, $\text{PE}_{2D}(r, c)$ will be identical to $\text{PE}_{2D}(c, r)$. For example, the positional encoding for patch $(1, 2)$ would be indistinguishable from that of patch $(2, 1)$ if $r \neq c$.

A common method for disentanglement is to allocate half the channels to the row index and half to the column index:

$$\text{PE}_{2D}(r, c) = [\text{PE}_{1D}^{\frac{d}{2}}(r) \parallel \text{PE}_{1D}^{\frac{d}{2}}(c)].$$

This removes the ambiguity present in the original additive answer.

Adding a learned [CLS] token: Pre-pend a learned vector $\mathbf{e}^{\text{CLS}} \in \mathbb{R}^d$ and (optionally) give it its own positional code $\mathbf{p}^{\text{CLS}}$ (often learned, sometimes zero):

$$\mathbf{z}_0 = \mathbf{e}^{\text{CLS}} + \mathbf{p}^{\text{CLS}}, \quad \mathbf{z}_{1+\ell} = \mathbf{x}_\ell + \text{PE}_{2D}(r_\ell, c_\ell),$$

where $\ell = 0, \dots, N-1$ enumerates the N image patches and $\mathbf{x}_\ell$ is the embedded patch content. Because [CLS] attends through self-attention like every other token, it can aggregate global information and feed it to the classification head.

(c) **Multi-variate time series**

The encoding $\text{PE}_{\text{signal}}(t, v) = \text{PE}_{1D}(t) + \mathbf{e}_v^{\text{var}}$ is a *sum of two terms that depend on disjoint variables*. A self-attention head can therefore, in principle, learn two separate linear projections, one that annihilates the variate part and one that annihilates the temporal part.

One drawback: because the two components are only *added*, interactions that are *multiplicative or phase-shifted* in nature cannot be expressed directly in the positional code.

(d) **Optional: Absolute vs. relative / rotary**

- **Absolute sinusoidal** encodings assign each position its own vector. For very deep trees or very long sequences, large indices compress into a narrow range of float magnitudes, making fine distinctions harder.
- Relative positional embeddings (Shaw et al.) and rotary positional encodings (RoPE) inject distance information directly into the attention scores. Because they depend on relative distances and use bounded operations, they extrapolate to longer sequences and avoid the extreme logits that can harm numerical stability.

Exercise 4: Compute Attention

Consider a single-head Transformer layer that processes a sequence of three token embeddings. The input matrix $X \in \mathbb{R}^{3 \times 2}$ contains these embeddings as row vectors (i.e., the first row is the embedding for token 1, the second row for token 2, and the third row for token 3):

$$X = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 2 & 1 \end{bmatrix}.$$

The projection matrices for queries, keys, and values are:

$$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_K = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad W_V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}.$$

Use the *scaled dot-product attention* formula:

$$\text{Att}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

where d_k is the dimension of the key vectors (and query vectors). The softmax function is applied row-wise to its input matrix. Keep at least four significant digits in intermediate calculations and your final answers.

- Project queries, keys and values** Compute the matrices $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. State the value of d_k .
- Raw compatibility scores** Compute the 3×3 scaled score matrix $S = \frac{QK^\top}{\sqrt{d_k}}$.
- Attention weights** Apply the row-wise softmax to S to obtain the attention weight matrix $A = \text{softmax}(S)$. Verify that each row of A sums to approximately 1 (accounting for potential rounding).
- Attention output** Compute the final output matrix $Z = AV \in \mathbb{R}^{3 \times 2}$.
- Causal masking** Repeat parts (b) – (d) using a *causal mask*. This means that before applying the softmax in part (c), you modify the scaled score matrix S (from part b) such that $S_{ij} = -\infty$ for all $j > i$ (i.e., a token cannot attend to subsequent tokens). Let this masked score matrix be S_{masked} . Then compute $A_{\text{causal}} = \text{softmax}(S_{\text{masked}})$ and $Z_{\text{causal}} = A_{\text{causal}}V$. Comment on how the output vectors in Z_{causal} for tokens 1, 2, and 3 differ from their counterparts in Z (from part d), and explain why these differences occur.

Solution

Let $X = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 2 & 1 \end{bmatrix}$, $W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $W_K = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & -0.5 \end{bmatrix}$, $W_V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$.

(a) $Q = XW_Q = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 2 & 1 \end{bmatrix}$. $K = XW_K = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & -0.5 \end{bmatrix} = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 0.5 \\ 1.5 & 0.5 \end{bmatrix}$. $V = XW_V = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 4 \\ 0 & -2 \\ 2 & 2 \end{bmatrix}$. Since $K \in \mathbb{R}^{3 \times 2}$, the dimension of key vectors $d_k = 2$.

$$(b) \quad K^\top = \begin{bmatrix} 1.5 & -0.5 & 1.5 \\ -0.5 & 0.5 & 0.5 \end{bmatrix}. \quad QK^\top = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1.5 & -0.5 & 1.5 \\ -0.5 & 0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 0.5 & 0.5 & 2.5 \\ 0.5 & -0.5 & -0.5 \\ 2.5 & -0.5 & 3.5 \end{bmatrix}.$$

$$\sqrt{d_k} = \sqrt{2} \approx 1.41421. \quad S = \frac{QK^\top}{\sqrt{d_k}} \approx \frac{1}{1.41421} \begin{bmatrix} 0.5 & 0.5 & 2.5 \\ 0.5 & -0.5 & -0.5 \\ 2.5 & -0.5 & 3.5 \end{bmatrix} \approx \begin{bmatrix} 0.35355 & 0.35355 & 1.76777 \\ 0.35355 & -0.35355 & -0.35355 \\ 1.76777 & -0.35355 & 2.47487 \end{bmatrix}.$$

Rounded to four significant digits for display:

$$S \approx \begin{bmatrix} 0.3536 & 0.3536 & 1.768 \\ 0.3536 & -0.3536 & -0.3536 \\ 1.768 & -0.3536 & 2.475 \end{bmatrix}.$$

(c) **Attention weights** Applying row-wise softmax to S :

$$e^{S_{1,:}} \approx [1.42415, 1.42415, 5.85854], \text{ sum } \approx 8.70684.$$

$$e^{S_{2,:}} \approx [1.42415, 0.70217, 0.70217], \text{ sum } \approx 2.82849.$$

$$e^{S_{3,:}} \approx [5.85854, 0.70217, 11.87989], \text{ sum } \approx 18.44060.$$

$$A = \text{softmax}(S) \approx \begin{bmatrix} 0.1636 & 0.1636 & 0.6729 \\ 0.5035 & 0.2483 & 0.2483 \\ 0.3177 & 0.03808 & 0.6442 \end{bmatrix}.$$

All rows sum to approximately 1.

(d) **Attention output**

$$Z = AV \approx \begin{bmatrix} 0.163570 & 0.163570 & 0.672861 \\ 0.503492 & 0.248254 & 0.248254 \\ 0.317700 & 0.038077 & 0.644223 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 0 & -2 \\ 2 & 2 \end{bmatrix} \approx \begin{bmatrix} 1.509 & 1.673 \\ 1.000 & 2.014 \\ 1.606 & 2.483 \end{bmatrix}.$$

(e) **Causal masking** Modify S from part (b) such that $S_{ij} = -\infty$ for $j > i$:

$$S_{\text{masked}} \approx \begin{bmatrix} 0.35355 & -\infty & -\infty \\ 0.35355 & -0.35355 & -\infty \\ 1.76777 & -0.35355 & 2.47487 \end{bmatrix}.$$

$A_{\text{causal}} = \text{softmax}(S_{\text{masked}})$. (Using $e^{-\infty} = 0$) Row 1: $e^{S_{1,:}} \approx [1.42415, 0, 0]$, sum ≈ 1.42415 . $A_{\text{causal},1} = [1, 0, 0]$. Row 2: $e^{S_{2,:}} \approx [1.42415, 0.70217, 0]$, sum ≈ 2.12632 . $A_{\text{causal},2} \approx [0.66977, 0.33023, 0]$. Row 3: $A_{\text{causal},3} \approx [0.31770, 0.03808, 0.64422]$.

$$A_{\text{causal}} \approx \begin{bmatrix} 1.000 & 0 & 0 \\ 0.6698 & 0.3302 & 0 \\ 0.3177 & 0.03808 & 0.6442 \end{bmatrix}.$$

$Z_{\text{causal}} = A_{\text{causal}}V$:

$$Z_{\text{causal}} \approx \begin{bmatrix} 1.000 & 0 & 0 \\ 0.669773 & 0.330227 & 0 \\ 0.317700 & 0.038077 & 0.644223 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 0 & -2 \\ 2 & 2 \end{bmatrix} \approx \begin{bmatrix} 1.000 & 4.000 \\ 0.6698 & 2.019 \\ 1.606 & 2.483 \end{bmatrix}.$$

Comments on differences between Z and Z_{causal} : Causal masking ensures a token's output depends only on current and past tokens, not future ones.

- **Token 1:** $Z_{\text{causal},1} = [1.000, 4.000]$ is identical to V_1 . This is because token 1 can only attend to itself ($A_{\text{causal},1} = [1, 0, 0]$). In contrast in the previous part, $Z = [1.509, 1.673]$ was a weighted average of V_1, V_2, V_3 .

- **Token 2:** $Z_{\text{causal},2} = [0.6698, 2.019]$ is a weighted average of V_1 and V_2 , as token 2 can only attend to tokens 1 and 2 ($A_{\text{causal},23} = 0$). $Z_2 = [1.000, 2.014]$ included attention to V_3 . The attention previously on token 3 is redistributed to tokens 1 and 2.
- **Token 3:** $Z_{\text{causal},3} = [1.606, 2.483]$ is identical to Z_3 . As token 3 is the last token in the sequence, its attention mechanism is not restricted by causal masking.